

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»**

Компьютерные науки и прикладная математика

УДК 519.6

СОГЛАСОВАНО

Научный руководитель,
Доктор физико-математических
наук, доцент

_____ А. Ю. Морозов
« ____ » _____ 2026 г.

УТВЕРЖДАЮ

Руководитель
образовательной программы

_____ С. С. Крылов
« ____ » _____ 2026 г.

**Выпускная квалификационная работа
(академическая)**

на тему: **Реализация параллельных алгоритмов интерполяции на основе
разреженных сеток с квадратичным базисом**

по направлению подготовки 02.03.02 «Фундаментальная информатика и
информационные технологии»

ВЫПОЛНИЛ

студент группы М8О-411Б-22
образовательной программы
02.03.02 «Фундаментальная
информатика и
информационные технологии»

_____ И. Н. Ивкович
« ____ » _____ 2026 г.

РЕФЕРАТ

Итоговая аттестационная работа состоит из 48 страниц, 4 глав, 9 рисунков, 7 листингов, 1 приложения. Использовано 10 источников.

РАЗРЕЖЕННЫЕ СЕТКИ; ИНТЕРПОЛЯЦИЯ; КВАДРАТИЧНЫЙ БАЗИС; ПАРАЛЛЕЛЬНЫЕ АЛГОРИТМЫ; ЧИСЛЕННЫЕ МЕТОДЫ; ДИФФЕРЕНЦИАЛЬНЫЕ УРАВНЕНИЯ; GO; FLUTTER

Объект разработки: программный комплекс для интерполяции многомерных отображений и решений дифференциальных уравнений на основе адаптивных разреженных сеток.

Цель работы: реализация и исследование параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом, а также создание прикладной системы для удалённого вычисления и визуализации результатов.

Методы и методология: методы численного анализа и интерполяции; иерархические базисные функции; адаптивное уточнение по коэффициентам избытка; использование контрольных точек; численное интегрирование систем обыкновенных дифференциальных уравнений методом Рунге–Кутты–Фельберга; методы параллельного программирования; клиент-серверная архитектура с использованием gRPC и protobuf.

Результаты работы и их новизна: разработано вычислительное ядро на языке Go, реализующее построение адаптивной разреженной сетки для скалярных и векторных отображений в последовательном и параллельном режимах. Реализована поддержка квадратичного базиса, позволяющего получать более гладкую аппроксимацию по сравнению с линейным вариантом. Создан серверный слой удалённого доступа, интерпретирующий пользовательские формулы и поддерживающий аппроксимацию операторов, возникающих при решении дифференциальных уравнений. Разработано клиентское приложение визуализации на Flutter. Научная новизна состоит в объединении адаптивной разреженной интерполяции, параллельного построения и прикладной инфраструктуры визуального анализа в рамках единого программного комплекса.

Область применения: визуализация и исследование двумерных динамических систем; численный эксперимент с операторами перехода; демонстрация методов адаптивной интерполяции в учебных курсах; использование как основы для дальнейших исследований в области параллельных численных методов.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	8
1 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА И МАТЕМАТИЧЕСКИЕ ОСНОВЫ МЕТОДА РАЗРЕЖЕННЫХ СЕТОК	11
1.1 Интерполяция многомерных функций и проблема роста вычислительной сложности	11
1.2 Нормировка области и постановка задачи интерполяции	11
1.3 Одномерные иерархические базисные функции	12
1.4 Многомерный базис и тензорное произведение	13
1.5 Иерархическое представление и коэффициенты избытка	13
1.6 Адаптивное уточнение и локальный критерий продолжения	13
1.7 Проблема вырожденного критерия на примере $\sin(x)$	14
1.8 Механизм anchor points как расширение базового метода	14
1.9 Порождение дочерних узлов	15
1.10 Жёсткое ограничение числа узлов как механизм оптимизации	15
1.11 Рекурсивное вычисление интерполянта	15
1.12 Связь с задачами моделирования динамических систем	16
Выводы по главе	16
2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И СТРУКТУР ДАННЫХ ПРОГРАММНОГО КОМПЛЕКСА	17
2.1 Роль вычислительного модуля в структуре проекта	17
2.2 Требования к вычислительному модулю	17
2.3 Центральный тип AdaptiveSparseGrid	18
2.4 Проектирование базовых структур данных	18
2.5 Архитектура построения сетки	19
2.6 Архитектура поддержки parallel и sequential build	19
2.7 Поддержка anchor points как архитектурное расширение	19
2.8 Жёсткое ограничение числа узлов как эксплуатационный параметр	20
2.9 Композиция отображений и метод следующей итерации	20
2.10 Поддержка дифференциальных уравнений на уровне архитектуры	20
2.11 Вспомогательный серверный слой	21
2.12 Клиентская визуализация как вспомогательный инструмент	21
2.13 Наблюдаемость и эксплуатационные характеристики	21
Выводы по главе	22

3 РЕАЛИЗАЦИЯ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ, АНАЛИЗ РЕЗУЛЬТАТОВ И ПОДГОТОВКА ИЛЛЮСТРАЦИЙ	23
3.1 Реализация вычислительного ядра на Go	23
3.2 Ключи узлов и кодирование положения в сетке	24
3.3 Реализация базисных функций	25
3.4 Параллельный и последовательный режимы построения	26
3.5 Создание узла и вычисление коэффициента избытка	27
3.6 Ключевой участок buildGrid: адаптация, anchor points и ограничение числа узлов	28
3.7 Рекурсивная интерполяция по построенной сетке	29
3.8 Два режима работы с дифференциальными уравнениями	30
3.9 Вспомогательный серверный слой	31
3.10 Клиентская визуализация	31
3.11 Анализ структуры бенчмарков	31
3.12 Анализ полученных данных	31
Выводы по главе	32
4 ОБОБЩЕНИЕ РЕЗУЛЬТАТОВ И НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ	33
4.1 Соответствие результатов поставленным задачам	33
4.2 Общая характеристика экспериментальной программы	33
4.3 Сравнение линейного и квадратичного базисов в Go-реализации	34
4.4 Сравнение parallel и sequential build в Go-реализации	35
4.5 Зависимость времени от размерности и размера области в Go-реализации	36
4.6 Влияние anchor points на производительность и корректность построения	37
4.7 Два подхода к работе с дифференциальными уравнениями в Go-реализации	37
4.8 Жёсткое ограничение числа узлов как средство оптимизации	38
4.9 Стоимость вычисления значения после построения	38
4.10 Четырёхмерная функция: сравнение всех сочетаний BuildType и BasisType	39
4.11 Сравнение Go- и C++-реализаций библиотеки	40
4.11.1 Сравнение последовательных реализаций	40
4.11.2 Сравнение параллельных реализаций	41
4.11.3 Сравнение режимов работы с дифференциальными уравнениями	41
4.11.4 Сравнение ограничения числа узлов	42
4.12 Преимущества разработанного решения	42
4.13 Ограничения текущей реализации	43
4.14 Иллюстрации двумерных функций, полученные во Flutter-клиенте	43
4.15 Перспективы дальнейшего развития	45
Выводы по главе	45
ЗАКЛЮЧЕНИЕ	46

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
--	----

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Flutter – кроссплатформенный UI-фреймворк, использованный для создания клиентского приложения визуализации

Go – компилируемый язык программирования, использованный в работе для реализации вычислительного ядра и серверного слоя

gRPC – фреймворк удалённого вызова процедур, использующий protobuf-контракты и HTTP/2 для обмена структурированными сообщениями между клиентом и сервером

Адаптивное уточнение – процесс локального увеличения детализации разреженной сетки в тех областях, где текущая аппроксимация оказывается недостаточно точной

Базисная функция – локальная функция, связанная с узлом сетки и определяющая вклад этого узла в итоговый интерполянт

Векторное поле – отображение, сопоставляющее каждой точке области вектор, например скорость изменения состояния динамической системы

Входная точка сетки (Entry Point) – начальный узел или стартовая конфигурация уровней и индексов, от которой начинается локальное построение адаптивной ветви разреженной сетки

Дифференциальное уравнение – уравнение, связывающее неизвестную функцию и её производные; в работе используется для задания динамики системы через правую часть

Иерархический коэффициент (коэффициент избытка) – разность между точным значением функции в узле и значением, восстановленным по уже построенной части сетки

Интерполянт – приближённое представление функции, позволяющее вычислять её значения по заранее построенной системе узлов и коэффициентов

Квадратичный базис – вариант базисной функции, в котором локальный вклад зависит квадратично от расстояния до центра узла; используется в работе как основной режим аппроксимации

Контрольная точка (Anchor Point) – заранее заданная точка области, в которой дополнительно проверяется ошибка интерполяции для принятия решения о продолжении адаптивного уточнения

Параллельный алгоритм – алгоритм, в котором независимые части вычислений выполняются одновременно на нескольких вычислительных потоках или ядрах процессора

Разреженная сетка (Sparse Grid) – иерархическая система узлов, предназначенная для аппроксимации многомерных функций с меньшим числом точек по сравнению с полной тензорной сеткой

Рекурсивный обход – способ обработки иерархической структуры данных, при котором вычисление в текущем узле дополняется вычислением во всех релевантных дочерних узлах

Тензорная сетка – полная многомерная сетка, получаемая декартовым произведением одномерных сеток; характеризуется быстрым ростом числа узлов с увеличением размерности

Численное интегрирование – приближённое вычисление решения дифференциального уравнения на конечном интервале времени с помощью дискретной схемы

ВВЕДЕНИЕ

Актуальность темы исследования

Во многих задачах вычислительной математики и моделирования требуется многократно вычислять значения сложных отображений на некоторой области параметров. Если исходная функция задаётся аналитически, но содержит дорогостоящие операции, либо определяется как результат численного интегрирования системы дифференциальных уравнений, прямой пересчёт в каждой точке становится вычислительно затратным. Особенно это проявляется в задачах визуализации фазовых портретов, исследования динамических систем, анализа векторных полей и построения операторов эволюции.

Одним из эффективных способов уменьшения вычислительных затрат является интерполяция на основе разреженных сеток [1; 2]. Разреженные сетки позволяют существенно сократить число узлов по сравнению с полной тензорной дискретизацией и при этом сохранить управляемую точность. Дополнительный интерес представляет адаптивное уточнение, при котором вычислительные ресурсы концентрируются только в областях сложного поведения функции. В сочетании с параллельной обработкой независимых задач такой подход делает возможным построение эффективных вычислительных модулей для интерактивных и серверных систем.

Степень разработанности проблемы

Теория разреженных сеток и иерархических базисов достаточно хорошо развита в численном анализе [1—3]. Для построения интерполянтов традиционно используются линейные базисные функции, однако в практических задачах аппроксимации гладких отображений более интересным оказывается квадратичный базис, способный лучше описывать локальную кривизну функции. Вопросы численного решения дифференциальных уравнений, в свою очередь, тесно связаны с задачей построения операторов перехода [4; 5].

В работе Морозова А. Ю. [6], использованной как одна из основных содержательных опор при выполнении дипломного проекта, рассматривается моделирование динамических систем с интервальными параметрами. Эта постановка естественным образом подводит к необходимости строить аппроксимации отображений, зависящих от неопределённых параметров, в том числе от времени. Однако практическая задача реализации параллельного вычислительного модуля, способного работать как с обычными отображениями, так и с операторами перехода, требует отдельного архитектурного и алгоритмического исследования.

Проблема исследования

При построении интерполяции для сложных отображений возникает противоречие между точностью и стоимостью вычислений. С одной стороны, повышение точности требует увеличения числа узлов и углубления адаптивного уточнения. С другой стороны, в прикладной системе время построения и стоимость последующего вычисления интерполанта должны оставаться ограниченными. Дополнительную сложность созда-

ёт то, что базовый адаптивный критерий может ошибочно остановить построение на функциях с осцилляциями и нулевыми значениями в ключевых точках. Следовательно, требуется метод и программная реализация, обеспечивающие баланс между точностью, скоростью, управляемостью размера сетки и устойчивостью адаптивного алгоритма.

Цель работы

Целью выпускной квалификационной работы является реализация и исследование параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом, а также создание вычислительного модуля, пригодного для аппроксимации как обычных отображений, так и решений дифференциальных уравнений.

Задачи исследования

Для достижения поставленной цели в работе были сформулированы и решены следующие задачи:

- 1) Исследовать математические основы иерархической интерполяции на разреженных сетках и сравнить линейный и квадратичный базисы.
- 2) Разработать структуры данных для представления адаптивной разреженной сетки, узлов, входных точек и коэффициентов избытка.
- 3) Реализовать вычислительный модуль, поддерживающий последовательный и параллельный режимы построения.
- 4) Разработать механизм anchor points для управления уточнением в случаях, когда базовый адаптивный критерий даёт ложный результат.
- 5) Реализовать возможность жёсткого ограничения числа узлов как дополнительный механизм оптимизации.
- 6) Реализовать два способа работы с дифференциальными уравнениями: через параметр времени как интервальную неопределённость и через итеративное применение `MakeNextIteration`.
- 7) Провести бенчмаркинг и проанализировать производительность разработанного решения в различных режимах работы.

Объект и предмет исследования

Объектом исследования являются методы интерполяции многомерных отображений и операторов перехода динамических систем на основе разреженных сеток.

Предметом исследования являются параллельные алгоритмы построения адаптивных разреженных сеток с квадратичным базисом, а также связанные с ними программные структуры данных и режимы вычислений.

Методы исследования

В работе использовались методы численного анализа, теории интерполяции, математического моделирования динамических систем, а также методы проектирования программного обеспечения. Для задач динамики использовалось численное интегрирование обыкновенных дифференциальных уравнений методом Рунге–Кутты–Фельберга

[4]. Для оценки эффективности применялись модульное тестирование и бенчмаркинг вычислительного модуля.

Научная новизна и практическая значимость

Научная новизна работы состоит в реализации и исследовании параллельного вычислительного модуля для адаптивной интерполяции на основе разреженных сеток с квадратичным базисом, а также в расширении базового подхода за счёт введения механизма `anchor points`, который позволяет вручную устранять ложную стабилизацию алгоритма на осциллирующих функциях. Дополнительным практически значимым результатом является реализация двух альтернативных способов работы с дифференциальными уравнениями и механизма жёсткого ограничения числа узлов.

Практическая значимость работы заключается в создании программного комплекса, ядром которого является модуль `compute`, пригодный для исследования динамических систем, численного эксперимента и визуализации структуры адаптивной сетки. Серверный слой и клиент визуализации расширяют область применения разработанного модуля и делают его удобным для удалённого использования.

Апробация результатов

Результаты работы не проходили апробацию в форме публикаций и докладов на конференциях.

1 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА И МАТЕМАТИЧЕСКИЕ ОСНОВЫ МЕТОДА РАЗРЕЖЕННЫХ СЕТОК

В первой главе рассматриваются математические основы интерполяции на разреженных сетках, лежащие в основе разработанного вычислительного модуля [1; 2]. Основное внимание уделяется иерархическому представлению аппроксимируемой функции, построению линейных и квадратичных базисных функций, критериям адаптивного уточнения и связи метода с моделированием динамических систем. Отдельно рассматривается предложенный в работе механизм *anchor points*, предназначенный для преодоления характерного вырожденного сценария адаптивной интерполяции.

1.1 Интерполяция многомерных функций и проблема роста вычислительной сложности

Пусть задано отображение

$$F: \Omega \subset \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad (1)$$

которое требуется многократно вычислять на области $\Omega = [a_1, b_1] \times \dots \times [a_d, b_d]$. Если каждое вычисление $F(x)$ связано с численным интегрированием, моделированием динамики или интерпретацией составной формулы, возникает задача построения приближённого интерполанта

$$\tilde{F}(x) \approx F(x), \quad x \in \Omega, \quad (2)$$

который обеспечивал бы контролируемую ошибку при существенно меньшей стоимости вычисления [3; 5].

Прямое построение полной тензорной сетки в d -мерном случае требует экспоненциального роста числа узлов. Если в каждом измерении используется n узлов, то общее число точек равно n^d . Такое поведение известно как «проклятие размерности». Поэтому вместо полной сетки применяются разреженные конструкции, в которых сохраняются только наиболее информативные комбинации одномерных уровней детализации [1; 2].

1.2 Нормировка области и постановка задачи интерполяции

В разработанной системе вычисления выполняются на единичном гиперкубе. Для этого исходная точка $x = (x_1, \dots, x_d)$ переводится из реальной области Ω в нормированные координаты $u = (u_1, \dots, u_d)$ по формуле

$$u_i = \frac{x_i - a_i}{b_i - a_i}, \quad i = 1, \dots, d. \quad (3)$$

Обратное преобразование имеет вид

$$x_i = a_i + u_i(b_i - a_i). \quad (4)$$

Такое нормирование позволяет единообразно задавать базисные функции и не связывать структуру метода с конкретным физическим масштабом задачи. В результате и математическая модель, и реализация модуля `compute` опираются на одну и ту же нормированную постановку.

1.3 Одномерные иерархические базисные функции

Построение разреженной сетки основано на иерархическом базисе. В одномерном случае каждому узлу сопоставляется уровень ℓ и индекс i . Для граничного уровня $\ell = 0$ используются две базисные функции, отвечающие концам отрезка:

$$\varphi_{0,0}(x) = 1 - x, \quad \varphi_{0,1}(x) = x. \quad (5)$$

Для уровней $\ell > 0$ положение центра узла определяется как

$$c_{\ell,i} = \frac{i}{2^\ell}, \quad (6)$$

а радиус влияния равен

$$h_\ell = \frac{1}{2^\ell}. \quad (7)$$

Если аргумент находится вне области влияния узла, вклад базисной функции равен нулю.

Для линейного базиса используется выражение

$$\varphi_{\ell,i}^{(lin)}(x) = 1 - \frac{|x - c_{\ell,i}|}{h_\ell}, \quad |x - c_{\ell,i}| < h_\ell. \quad (8)$$

Для квадратичного базиса, который является основным в данной работе, применяется функция

$$\varphi_{\ell,i}^{(quad)}(x) = 1 - \left(\frac{|x - c_{\ell,i}|}{h_\ell} \right)^2, \quad |x - c_{\ell,i}| < h_\ell. \quad (9)$$

Квадратичный базис обеспечивает более гладкое поведение интерполянта внутри области влияния узла и лучше согласуется с аппроксимацией гладких отображений и операторов динамики, чем линейный вариант [3; 7].

1.4 Многомерный базис и тензорное произведение

Для точки $u = (u_1, \dots, u_d)$ вклад узла с уровнями $\ell = (\ell_1, \dots, \ell_d)$ и индексами $i = (i_1, \dots, i_d)$ задаётся произведением одномерных функций:

$$\Phi_{\ell,i}(u) = \prod_{k=1}^d \varphi_{\ell_k, i_k}(u_k). \quad (10)$$

Такое определение соответствует тензорному произведению одномерных базисов. Если хотя бы в одном измерении аргумент выходит из области влияния соответствующего узла, весь многомерный базис обращается в нуль. Именно это свойство обеспечивает локальность интерполяции и делает рекурсивный обход дерева вычислительно эффективным.

1.5 Иерархическое представление и коэффициенты избытка

Интерполянт на разреженной сетке строится в виде суммы иерархических вкладов:

$$\tilde{F}(u) = \sum_{(\ell,i) \in \mathcal{G}} \alpha_{\ell,i} \Phi_{\ell,i}(u), \quad (11)$$

где $\mathcal{G}$ — множество узлов сетки, а $\alpha_{\ell,i} \in \mathbb{R}^m$ — иерархические коэффициенты, или коэффициенты избытка [1].

Если обозначить через $u_{\ell,i}$ координаты центра узла, то коэффициент вычисляется как разность между точным значением и уже имеющейся интерполяцией:

$$\alpha_{\ell,i} = F(u_{\ell,i}) - \tilde{F}_{prev}(u_{\ell,i}). \quad (12)$$

Если норма коэффициента мала, это означает, что в данной точке текущее приближение уже хорошо описывает поведение функции. Наоборот, большие значения $\|\alpha_{\ell,i}\|$ указывают на необходимость дальнейшего уточнения.

1.6 Адаптивное уточнение и локальный критерий продолжения

В работе используется адаптивная стратегия построения сетки. Решение о создании дочерних узлов принимается по величине коэффициента избытка и по дополнительной проверке в специальных точках. Базовый критерий имеет вид

$$\|\alpha_{\ell,i}\| \leq \varepsilon, \quad (13)$$

где ε — допустимая погрешность.

Однако практика показывает, что опора только на значение коэффициента в центре узла может приводить к ложному завершению алгоритма. Этот эффект особенно

ярко проявляется на осциллирующих функциях.

1.7 Проблема вырожденного критерия на примере $\sin(x)$

Для адаптивных сеток существует характерная проблема. Рассмотрим функцию

$$f(x) = \sin(x) \quad (14)$$

на интервале $[0, 4\pi]$. На краях этого интервала и в его центре функция обращается в нуль:

$$f(0) = 0, \quad f(2\pi) = 0, \quad f(4\pi) = 0. \quad (15)$$

Если алгоритм анализирует только эти точки, он может сделать ложный вывод, что функция близка к прямой линии, то есть что дополнительное уточнение не требуется. Но фактически внутри интервала находятся выраженные экстремумы и осцилляции, которые не видны по такой редкой выборке.

Именно эта ситуация выявилась в ходе работы над дипломом. Для её устранения был реализован специальный механизм *anchor points*, отсутствовавший в исходной статье научного руководителя [6]. Он позволяет вручную задать точки, в которых алгоритм обязан дополнительно контролировать ошибку и при необходимости продолжать уточнение сетки.

1.8 Механизм *anchor points* как расширение базового метода

Пусть внутри области влияния текущего узла имеется контрольная точка $u^{(a)}$. Тогда дополнительно проверяется условие

$$\left\| F(u^{(a)}) - \tilde{F}(u^{(a)}) \right\| < \varepsilon. \quad (16)$$

Если оно нарушается, алгоритм обязан продолжать построение дочерних узлов, даже если центральный коэффициент избытка оказался малым.

Таким образом, *anchor points* выполняют две функции:

- 1) предотвращают ложное «схлопывание» сетки на функциях с нулями и симметрией;
- 2) позволяют пользователю вручную указывать области, которые обязательно должны быть разрешены более подробно.

С математической точки зрения этот механизм можно интерпретировать как введение дополнительного множества ограничений на локальную ошибку. С практической точки зрения он является важным расширением базового алгоритма и одним из собственных результатов работы.

1.9 Порождение дочерних узлов

Пусть текущий узел имеет уровень $\ell_k > 0$ в k -м измерении. Тогда при уточнении по этой координате строятся два дочерних узла с уровнем

$$\ell'_k = \ell_k + 1, \quad (17)$$

и индексами

$$i_k^{left} = 2i_k - 1, \quad i_k^{right} = 2i_k + 1. \quad (18)$$

Все остальные уровни и индексы сохраняются. Такая формула отражает стандартное иерархическое ветвление.

В реализации дополнительно используется информация о направлении ошибки: если контрольная точка лежит слева от центра узла, приоритет отдаётся левой ветви, и наоборот. Это уменьшает число лишних вычислений.

1.10 Жёсткое ограничение числа узлов как механизм оптимизации

Помимо ограничения по точности и глубине уточнения, в работе реализована возможность жёсткого ограничения числа узлов. Если обозначить допустимый предел через N_{max} , то построение может быть принудительно остановлено при выполнении условия

$$|\mathcal{G}| \geq N_{max}. \quad (19)$$

Этот механизм важен с практической точки зрения. Он позволяет заранее ограничивать объём памяти и время построения сетки для сложных задач, когда пользователь предпочитает контролировать ресурсные затраты. Таким образом, ограничение числа узлов выступает дополнительным средством оптимизации наряду с выбором ε , типа базиса и максимального уровня.

1.11 Рекурсивное вычисление интерполянта

После построения сетки значение интерполянта в точке u вычисляется как сумма вкладов входных узлов и их потомков. Для узла n с коэффициентом α_n и базисной функцией Φ_n локальный вклад равен

$$V_n(u) = \alpha_n \Phi_n(u). \quad (20)$$

Если узел имеет потомков, добавляются их рекурсивные вклады:

$$S_n(u) = V_n(u) + \sum_{c \in \text{Children}(n)} S_c(u). \quad (21)$$

Локальность базисных функций обеспечивает быстрое завершение рекурсии: если $\Phi_n(u) = 0$, вклад всей соответствующей ветви равен нулю.

1.12 Связь с задачами моделирования динамических систем

Пусть динамическая система задаётся дифференциальным уравнением

$$\frac{dz}{dt} = f(z, t), \quad z(t_0) = z_0. \quad (22)$$

Тогда оператор перехода на временном шаге Δt можно записать как отображение

$$\Psi_{\Delta t}(z_0) = z(t_0 + \Delta t). \quad (23)$$

Подход, использованный в работе, позволяет интерполировать либо сам оператор перехода, либо его последовательные композиции. Это напрямую соотносится с тематикой моделирования динамических систем с интервальными параметрами, рассматриваемой в работе Морозова А. Ю. [6]. В дипломном проекте эта идея получила программное воплощение в модуле `compute` и была распространена на поддержку двух режимов работы со временем: через включение параметра t как дополнительной размерности и через итеративное применение одношагового оператора.

Выводы по главе

В первой главе рассмотрены математические основы метода, используемого в дипломной работе. Показано, что переход к разреженным сеткам позволяет уменьшить число узлов по сравнению с полной тензорной дискретизацией, а иерархическое представление на базе коэффициентов избытка обеспечивает естественный механизм адаптивного уточнения. Обосновано использование квадратичного базиса как более точного инструмента для аппроксимации гладких отображений. Отдельно описана проблема вырожденного критерия на функции $\sin(x)$ и предложенный в работе механизм `anchor points`, позволяющий вручную принудить алгоритм уточнять сетку в значимых областях. Также показано, что аппарат разреженных сеток применим не только к обычным функциям, но и к операторам перехода, возникающим при численном решении дифференциальных уравнений. Эти положения образуют теоретическую базу для архитектуры и алгоритмов, реализованных в вычислительном модуле проекта.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И СТРУКТУР ДАННЫХ ПРОГРАММНОГО КОМПЛЕКСА

Во второй главе рассматривается архитектура разработанного программного комплекса. Основной акцент сделан на вычислительном модуле `compute`, поскольку именно он является центральным результатом дипломной работы. Серверный слой на gRPC и клиент визуализации на Flutter рассматриваются как вспомогательные компоненты, разработанные для удобства экспериментов, удалённого запуска и анализа результатов.

2.1 Роль вычислительного модуля в структуре проекта

Целью дипломной работы является реализация параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом. Этой цели непосредственно соответствует модуль `compute`, в котором сосредоточены все ключевые математические и алгоритмические решения: представление узлов, вычисление базисных функций, построение адаптивной сетки, рекурсивная интерполяция, композиция операторов и поддержка режимов последовательного и параллельного построения.

Сервисный слой и пользовательский интерфейс не определяют математическое содержание работы, а лишь расширяют область практического применения созданного ядра. По этой причине архитектура программного комплекса должна рассматриваться прежде всего как архитектура вычислительного модуля, дополненная средствами визуализации и сетевого доступа.

2.2 Требования к вычислительному модулю

К модулю `compute` были предъявлены следующие требования:

- 1) поддержка многомерных входных данных и векторных выходных значений;
- 2) поддержка линейного и квадратичного базисов;
- 3) адаптивное уточнение сетки на основе коэффициентов избытка;
- 4) возможность учёта `anchor points` для ручного управления уточнением;
- 5) поддержка последовательного и параллельного режимов построения;
- 6) возможность жёсткого ограничения числа узлов;
- 7) поддержка композиции отображений и последовательных итераций оператора;
- 8) совместимость с задачами прямой интерполяции и численного решения дифференциальных уравнений.

Именно выполнение этих требований определяет архитектурные решения, рассмотренные далее.

2.3 Центральный тип `AdaptiveSparseGrid`

Ключевой структурой вычислительного модуля является тип `AdaptiveSparseGrid`. Он объединяет в одном объекте:

- 1) границы области определения;
- 2) тип используемого базиса;
- 3) размерность входного и выходного пространств;
- 4) набор входных точек построения;
- 5) словарь всех узлов сетки.

Такое проектирование позволяет трактовать объект сетки одновременно как:

- 1) контейнер данных, описывающий уже построенный интерполянт;
- 2) исполняемый оператор, умеющий вычислять приближённое значение в новой точке;
- 3) основу для построения следующей итерации или композиции отображений.

Это архитектурное свойство особенно важно для задач моделирования динамических систем, где один и тот же объект должен не только хранить аппроксимацию, но и участвовать в последующих шагах вычисления.

2.4 Проектирование базовых структур данных

Для реализации метода были введены следующие типы.

Тип `Point`. Представляет собой вектор вещественных координат. Он используется как для входных точек, так и для значений функции и коэффициентов избытка. Использование единого векторного представления позволяет поддерживать как скалярные, так и векторные отображения без дублирования алгоритмов.

Тип `gridKey`. Определяет положение узла в иерархической структуре через массивы уровней и индексов. Такой ключ задаёт не только идентификатор узла, но и его геометрический смысл: уровень отвечает за масштаб, индекс — за положение центра.

Тип `node`. Содержит ключ, координаты центра в единичной области, коэффициент избытка и признак наличия потомков. Через эту структуру материализуется математическая модель адаптивной сетки.

Тип `entryPoint`. Описывает стартовую конфигурацию построения. Входные точки играют важную роль, поскольку именно от них запускаются независимые локальные ветви построения, что напрямую связано с возможностью распараллеливания.

В совокупности эти структуры образуют минимальное, но достаточное ядро вычислительной модели.

2.5 Архитектура построения сетки

С точки зрения архитектуры модуль `compute` организован вокруг процедуры построения, которая разделяется на два уровня.

На первом уровне формируются стартовые конфигурации уровней и индексов, соответствующие различным входным точкам. На втором уровне для каждой такой конфигурации запускается локальная задача адаптивного построения. Локальная задача:

- 1) создаёт начальный узел;
- 2) вычисляет коэффициент избытка;
- 3) проверяет критерии продолжения;
- 4) при необходимости порождает дочерние узлы;
- 5) рекурсивно или итеративно расширяет сетку до выполнения ограничений.

Это решение позволяет отделить глобальную организацию стартовых конфигураций от локальной логики уточнения. Такое разделение удобно и математически, и программно: оно делает код более модульным и упрощает последующую параллелизацию.

2.6 Архитектура поддержки `parallel` и `sequential build`

В модуле предусмотрены два режима работы: `BuildTypeSequential` и `BuildTypeParallel`. Их поддержка реализована не как два разных алгоритма, а как два режима исполнения одной и той же логики построения.

В последовательном режиме все локальные задачи выполняются по очереди в одном потоке. Этот режим удобен для проверки корректности, профилирования и сопоставления с параллельным вариантом.

В параллельном режиме независимые стартовые конфигурации обрабатываются одновременно. Архитектурное достоинство такого подхода состоит в том, что конкурентность не «вшивается» в математические операции над узлом, а накладывается поверх уже выделенных независимых задач. Благодаря этому ядро сохраняет логическую простоту, а параллелизация не разрушает его структуру.

2.7 Поддержка `anchor points` как архитектурное расширение

В ходе разработки была выявлена проблема, когда адаптивный критерий, опирающийся только на коэффициент избытка в центре узла, может преждевременно остановить уточнение. Для решения этой проблемы в архитектуру вычислительного модуля был включён дополнительный механизм `anchor points`.

С архитектурной точки зрения `anchor points` представляют собой внешний набор контрольных точек, передаваемых в конструктор сетки. Они не изменяют базовую формулу интерполяции, но влияют на принятие решения о продолжении построения. Это

хороший пример расширения алгоритма без изменения его фундаментальной структуры: в ядро добавляется дополнительный слой условий, не нарушающий основной логики вычисления коэффициентов и рекурсивного обхода.

2.8 Жёсткое ограничение числа узлов как эксплуатационный параметр

Важным архитектурным решением является поддержка параметра `maxNodesInGrid`. Он задаёт верхнюю границу числа узлов, после которой построение должно завершиться принудительно. Наличие этого ограничения позволяет использовать вычислительный модуль не только в исследовательском режиме, но и в условиях ограниченных ресурсов.

С практической точки зрения это означает, что пользователь может выбрать один из двух режимов управления сложностью:

- 1) через точность и глубину уточнения;
- 2) через жёсткий лимит на размер сетки.

Такое решение особенно важно для серверного использования, где необходимо контролировать время ответа и объём данных, передаваемых клиенту.

2.9 Композиция отображений и метод следующей итерации

Одной из сильнейших сторон архитектуры модуля `compute` является поддержка композиции отображений через метод `MakeNextIteration`. В отличие от обычного вычислительного объекта, сетка в этом случае используется как промежуточный оператор: её результат подаётся на вход следующей функции, после чего строится новая аппроксимация.

С архитектурной точки зрения это превращает `AdaptiveSparseGrid` в универсальный вычислительный кирпич, пригодный для последовательного моделирования шагов эволюции. Именно поэтому модуль `compute` подходит не только для статической интерполяции, но и для задач динамики.

2.10 Поддержка дифференциальных уравнений на уровне архитектуры

Для работы с дифференциальными уравнениями используются два концептуально разных режима.

1) **Параметр t как интервальная неопределённость.** В этом случае время включается как дополнительная координата входного пространства. Тогда аппроксимируется отображение, зависящее как от начального состояния, так и от значения времени.

2) **Итеративный режим через `MakeNextIteration`.** В этом случае строится одношаговый оператор перехода, после чего он последовательно композиционно применяется несколько раз.

Архитектурная ценность этой двойной схемы состоит в том, что один и тот же вычислительный модуль способен поддерживать два разных численных сценария без изменения своего интерфейса. Это также делает возможным содержательное сравнение их производительности на уровне бенчмарков.

2.11 Вспомогательный серверный слой

Слой `internal/server` не является центральным результатом дипломной работы, но играет важную вспомогательную роль. Его задача — предоставить удобный удалённый интерфейс к вычислительному модулю. Сервер:

- 1) принимает параметры задачи;
- 2) интерпретирует пользовательские формулы;
- 3) при необходимости решает дифференциальное уравнение численным методом;
- 4) передаёт полученное отображение в `compute`;
- 5) сериализует построенную сетку в `protobuf`-ответ.

Таким образом, серверный слой следует рассматривать как адаптер между пользовательским запросом и вычислительным ядром, а не как самостоятельный предмет исследования.

2.12 Клиентская визуализация как вспомогательный инструмент

Аналогичную роль играет и Flutter-приложение. Оно предназначено для:

- 1) ввода параметров задачи;
- 2) запуска расчётов без работы из командной строки;
- 3) визуального анализа распределения узлов;
- 4) демонстрации результатов в наглядной форме.

Клиентская часть важна для практического применения проекта, однако архитектурный центр диплома находится именно в модуле `compute`. Flutter-интерфейс выступает как средство визуализации результатов работы этого модуля.

2.13 Наблюдаемость и эксплуатационные характеристики

Несмотря на вспомогательную роль серверного слоя, в проекте предусмотрены средства эксплуатационного мониторинга: метрики Prometheus, контроль длительности запроса, числа узлов и размера ответа. Эти механизмы полезны прежде всего для

оценки того, как параметры вычислительного модуля отражаются на внешних характеристиках системы.

Иначе говоря, наблюдаемость сервиса введена не ради самой серверной архитектуры, а как средство исследования поведения алгоритмов, реализованных в `compute`.

Выводы по главе

Во второй главе была рассмотрена архитектура программного комплекса с акцентом на вычислительный модуль `compute`, являющийся главным результатом дипломной работы. Показано, что именно в этом модуле сосредоточены все ключевые математические и алгоритмические решения: представление узлов, адаптивное уточнение, поддержка `anchor points`, ограничение числа узлов, последовательный и параллельный режимы построения, а также композиция операторов и работа с дифференциальными уравнениями. Серверный слой на `gRPC` и клиент визуализации на `Flutter` рассмотрены как вспомогательные средства удалённого доступа и анализа результатов. Такая архитектура подчёркивает, что ядром проекта является именно реализация параллельных алгоритмов интерполяции на основе разреженных сеток.

3 РЕАЛИЗАЦИЯ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ, АНАЛИЗ РЕЗУЛЬТАТОВ И ПОДГОТОВКА ИЛЛЮСТРАЦИЙ

В третьей главе рассматривается практическая реализация вычислительного модуля и его интеграция с вспомогательными компонентами проекта. Основной акцент сделан на устройстве пакета `compute`, поскольку именно он реализует математический аппарат адаптивных разреженных сеток, поддержку линейного и квадратичного базисов, параллельное построение, контроль числа узлов и механизмы работы с динамическими системами.

3.1 Реализация вычислительного ядра на Go

Основная часть алгоритмов сосредоточена в пакете `compute`. Его центральным элементом является тип `AdaptiveSparseGrid`, инкапсулирующий параметры области, тип базиса, размерности входа и выхода, список входных точек и множество узлов. Выбор Go в качестве языка реализации вычислительного модуля обусловлен поддержкой конкурентных вычислений, удобной работой с функциями высшего порядка и простой интеграции в прикладной сервис [8].

Конструктор сетки выполняет валидацию входных данных, определяет размерность образа функции, вычисляет значения в контрольных точках и затем запускает построение. Важной особенностью является то, что вычислительное ядро принимает пользовательскую функцию как аргумент. Благодаря этому один и тот же код используется и для интерполяции аналитических отображений, и для операторов перехода, получаемых численным интегрированием.

В листинге 1 показан фрагмент конструктора, задающего общий сценарий создания сетки.

Листинг 1 — Создание адаптивной разреженной сетки

```
1 func NewAdaptiveSparseGridWithContext(  
2     ctx context.Context,  
3     funcEval func(Point) (Point, error),  
4     min, max Point,  
5     epsilon ScalarType,  
6     anchorPoints []Point,  
7     basisType BasisType,  
8     buildType BuildType,  
9     maxLevel int64,  
10    maxNodesInGrid int64,  
11 ) (*AdaptiveSparseGrid, error) {  
12     inDim := int64(len(min))  
13     if int64(len(max)) != inDim {  
14         return nil, fmt.Errorf("min and max must have the same dimension")
```

```

15     }
16
17     sampleOut, err := funcEval(min)
18     if err != nil {
19         return nil, err
20     }
21     outDim := int64(len(sampleOut))
22
23     grid := &AdaptiveSparseGrid{
24         min:      min,
25         max:      max,
26         basisType: basisType,
27         inDim:    inDim,
28         outDim:   outDim,
29         nodes:    make(map[string]node),
30     }
31
32     anchors, err := grid.calculateOriginalFunctionAtPoints(funcEval, anchorPoints)
33     if err != nil {
34         return nil, err
35     }
36
37     err = grid.build(ctx, funcEval, epsilon, anchors, buildType, maxLevel,
38         maxNodesInGrid)
39     if err != nil {
40         return nil, err
41     }
42     return grid, nil
43 }

```

В данном фрагменте видны основные свойства реализации: независимость от природы исходной функции, автоматическое определение размерности результата и наличие параметров, отвечающих за точность, тип базиса, режим построения и жёсткое ограничение числа узлов.

3.2 Ключи узлов и кодирование положения в сетке

Каждый узел сетки идентифицируется набором уровней и индексов. Для хранения узлов в отображении используется тип `gridKey`. Его строковое представление формируется бинарной упаковкой целочисленных массивов.

В листинге 2 показан соответствующий код.

Листинг 2 — Представление ключа узла сетки

```

1 type gridKey struct {
2     level []int64
3     index []int64
4 }

```



```

5
6 func (k gridKey) String() string {
7     totalLength := (len(k.level) + len(k.index)) * 8
8     result := make([]byte, totalLength)
9
10    pos := 0
11    for _, v := range k.level {
12        binary.LittleEndian.PutUint64(result[pos:], uint64(v))
13        pos += 8
14    }
15    for _, v := range k.index {
16        binary.LittleEndian.PutUint64(result[pos:], uint64(v))
17        pos += 8
18    }
19
20    return unsafe.String(&result[0], len(result))
21 }

```

Достоинство этого решения состоит в быстром поиске узла по его иерархическому положению. Это критически важно для построения потомков и для рекурсивной интерполяции по дереву узлов.

3.3 Реализация базисных функций

Математическая модель из первой главы непосредственно отражается в коде вычисления одномерного базиса. В зависимости от уровня, индекса и выбранного типа базиса вычисляется значение локальной функции влияния. Именно здесь программно различаются линейный и квадратичный варианты.

В листинге 3 приведена функция вычисления одномерного базиса.

Листинг 3 — Вычисление одномерной базисной функции

```

1 func eval1d(x float64, level int64, index int64, basisType BasisType) float64 {
2     if level == 0 {
3         if index == 0 {
4             return 1.0 - x
5         }
6         if index == 1 {
7             return x
8         }
9         return 0.0
10    }
11
12    h := 1.0 / float64(int64(1)<<level)
13    center := float64(index) * h
14    dist := math.Abs(x - center)
15
16    if dist >= h {
17        return 0.0

```

```

18     }
19
20     t := dist / h
21     switch basisType {
22     case BasisTypeLinear:
23         return 1.0 - t
24     case BasisTypeQuadratic:
25         return 1.0 - t*t
26     }
27     return 0.0
28 }

```

Данный фрагмент важен не только как реализация базиса, но и как точка прямого сопоставления двух режимов работы, сравнение которых затем проводится в бенчмарках.

3.4 Параллельный и последовательный режимы построения

Поддержка двух режимов сборки реализована без раздвоения математической логики. Независимые стартовые конфигурации либо вычисляются по очереди, либо запускаются как отдельные goroutine. На практике это позволяет сравнивать режимы `parallel` и `sequential` на одном и том же наборе задач.

В листинге 4 приведён фрагмент функции `build`, показывающий выбор между последовательным и параллельным режимами.

Листинг 4 — Параллельный запуск локальных задач построения

```

1  if buildType == BuildTypeParallel {
2      wg.Add(1)
3      go func(k gridKey) {
4          defer wg.Done()
5          res := g.buildGridTask(ctx, funcEval, epsilon, anchors, k, i, maxLevel,
6                                  maxNodesInGrid)
7          mu.Lock()
8          results = append(results, res)
9          mu.Unlock()
10     }(currentKey)
11 } else {
12     res := g.buildGridTask(ctx, funcEval, epsilon, anchors, currentKey, i, maxLevel,
13                             , maxNodesInGrid)
14     results = append(results, res)
15 }

```

Такой способ организации вычислений хорошо согласуется с логикой метода, поскольку параллелизуются именно независимые задачи первого уровня.

3.5 Создание узла и вычисление коэффициента избытка

При создании нового узла вычисляется значение исходной функции, затем значение текущей интерполяции в той же точке, после чего формируется коэффициент избытка как их разность.

В листинге 5 показан соответствующий фрагмент кода.

Листинг 5 — Формирование узла и вычисление коэффициента избытка

```
1 func (g *AdaptiveSparseGrid) createNode(  
2     funcEval func(Point) (Point, error),  
3     key gridKey,  
4     entryPoint *node,  
5     dimension int64,  
6     additionalNodes map[string]node,  
7 ) (*gridKey, error) {  
8     if _, exists := additionalNodes[key.String()]; exists {  
9         return nil, nil  
10    }  
11  
12    n := node{  
13        key:      key,  
14        centerUnit: make(Point, g.inDim),  
15    }  
16    for i := int64(0); i < g.inDim; i++ {  
17        n.centerUnit[i] = getCoord(key.level[i], key.index[i])  
18    }  
19  
20    realArg := g.toReal(n.centerUnit)  
21    etalon, err := funcEval(realArg)  
22    if err != nil {  
23        return nil, err  
24    }  
25  
26    interp := g.evaluateForDimAndEntryPoint(n.centerUnit, dimension, entryPoint,  
27        additionalNodes)  
28    etalon.Sub(interp)  
29    n.alpha = etalon  
30    additionalNodes[key.String()] = n  
31  
32    return &key, nil  
33 }
```

Этот код непосредственно реализует иерархическую природу метода: узел хранит не абсолютное значение функции, а именно добавочную информацию относительно уже построенного приближения.

3.6 Ключевой участок buildGrid: адаптация, anchor points и ограничение числа узлов

Наиболее важным фрагментом реализации является внутренняя логика buildGrid, где одновременно решаются задачи анализа коэффициента избытка, учёта anchor points, порождения дочерних узлов и контроля числа узлов. Именно этот участок лучше всего отражает реальное содержание вычислительного модуля, поэтому он является более показательным, чем элементарные операции над типом Point.

В листинге 6 приведён характерный фрагмент алгоритма.

Листинг 6 — Фрагмент алгоритма buildGrid: критерии продолжения и ограничение числа узлов

```
1 | for len(nodeQueue) > 0 {
2 |     currentKey := nodeQueue[0]
3 |     nodeQueue = nodeQueue[1:]
4 |
5 |     currentNode := newNodes[currentKey.String()]
6 |     activeEntryNode := newNodes[entryPointKeyString]
7 |
8 |     canContinue := false
9 |     directions := make([]direction, g.inDim)
10 |    for i := range directions {
11 |        directions[i] = directionBoth
12 |    }
13 |
14 |    if currentNode.alpha.Length() <= epsilon {
15 |        for i := range directions {
16 |            directions[i] = directionNone
17 |        }
18 |        canContinue = true
19 |    }
20 |
21 |    if canContinue {
22 |        for _, anchor := range anchors {
23 |            if currentNode.isPointInAffectZone(anchor.arg, g.inDim) {
24 |                evalRes := g.evaluateForDimAndEntryPoint(anchor.arg, dimension, &
25 |                    activeEntryNode, newNodes)
26 |                evalRes.Sub(anchor.ans)
27 |                if evalRes.Length() >= epsilon {
28 |                    canContinue = false
29 |                    affectDirs := getAffectDirection(anchor.arg, currentNode.key.
30 |                        level, currentNode.key.index, g.inDim)
31 |                    for i := int64(0); i < g.inDim; i++ {
32 |                        directions[i] |= affectDirs[i]
33 |                    }
34 |                }
35 |            }
36 |        }
37 |    }
38 | }
```

```

34     }
35 }
36
37 if canContinue {
38     continue
39 }
40
41 currentNode.hasChildren = true
42 newNodes[currentKey.String()] = currentNode
43
44 if maxNodesInGrid > 0 && int64(len(newNodes)) >= maxNodesInGrid {
45     break
46 }
47 }

```

Этот фрагмент отражает несколько существенных особенностей реализации:

- 1) базовое решение принимается по норме коэффициента избытка;
- 2) anchor points могут отменить преждевременную остановку;
- 3) уточнение выполняется направленно;
- 4) рост сетки может быть жёстко ограничен параметром `maxNodesInGrid`.

Именно здесь проявляется одно из главных отличий реализованного модуля от базовой математической схемы, рассматриваемой в статье [6]: введение ручного механизма управления уточнением через anchor points.

3.7 Рекурсивная интерполяция по построенной сетке

После завершения построения объект `AdaptiveSparseGrid` используется для быстрого вычисления приближённых значений. Интерполяция организована как рекурсивный обход дерева узлов: сначала учитывается вклад текущего узла, затем — вклад релевантных потомков.

В листинге 7 показан характерный фрагмент рекурсивного вычисления.

Листинг 7 — Рекурсивное вычисление интерполянта

```

1 func (g *AdaptiveSparseGrid) evaluateRecursive(
2     x Point,
3     n node,
4     processedNodes map[string]struct{},
5     additionalNodes map[string]node,
6 ) Point {
7     if _, exists := processedNodes[n.key.String()]; exists {
8         return make(Point, g.outDim)
9     }
10
11     processedNodes[n.key.String()] = struct{}{}
12     basis := evalBasis(x, n.key.level, n.key.index, g.basisType, g.inDim)
13

```

```

14     if math.Abs(basis) < 1e-9 {
15         return make(Point, g.outDim)
16     }
17
18     value := n.alpha.Clone()
19     value.MulScalar(basis)
20
21     if n.hasChildren {
22         for i := int64(0); i < g.inDim; i++ {
23             if n.key.level[i] != 0 {
24                 child, exists := g.getChildForDimAndArg(n, x, i, additionalNodes)
25                 if exists {
26                     value.Add(g.evaluateRecursive(x, child, processedNodes,
27                                             additionalNodes))
28                 }
29             }
30         }
31     }
32     return value
33 }

```

Программный смысл этого кода совпадает с теоретической формулой суммы иерархических вкладов. Если значение базиса практически нулевое, соответствующая ветвь обхода мгновенно отсекается.

3.8 Два режима работы с дифференциальными уравнениями

Для задач динамики в проекте поддерживаются два принципиально разных подхода.

Первый подход: параметр t как интервальная неопределённость. В этом случае время добавляется как ещё одна координата пространства аргументов. Тогда строится сетка для отображения

$$(x_0, t) \mapsto x(t). \quad (24)$$

Преимущество этого подхода состоит в том, что одна построенная сетка сразу описывает поведение решения на целом временном интервале. Недостаток — рост размерности задачи и, как следствие, рост вычислительной сложности.

Второй подход: итеративное построение через `MakeNextIteration`. В этом случае сначала строится одношаговый оператор перехода на интервале длины 1, а затем он применяется последовательно несколько раз. Преимущество такого подхода состоит в том, что размерность пространства не увеличивается, но приходится выполнять серию последовательных построений.

Именно сопоставление этих двух подходов является одним из ключевых содержа-

тельных результатов экспериментальной части, поскольку оно показывает, что разные трактовки времени приводят к разным профилям вычислительной сложности.

3.9 Вспомогательный серверный слой

Пакет `internal/server` реализует удалённый интерфейс к вычислительному ядру. Объект `Formula` использует библиотеку `expr` для интерпретации пользовательских выражений [9]. При работе с дифференциальными уравнениями сервер численно интегрирует систему методом Рунге–Кутты–Фельберга [4], после чего передаёт в модуль `compute` уже готовое отображение.

С точки зрения архитектуры серверный слой является адаптером, а не предметом основной новизны работы. Его значение состоит в том, что он позволяет использовать вычислительный модуль удалённо и удобно подключать визуализацию.

3.10 Клиентская визуализация

Клиентская часть на Flutter реализует ввод параметров, запуск вычислений и отображение узлов сетки [10]. Она важна как средство анализа результатов, но не является математическим ядром работы. Основной результат диплома по-прежнему сосредоточен в модуле `compute`; клиент выступает как инструмент демонстрации и визуального исследования структуры построенного интерполянта.

3.11 Анализ структуры бенчмарков

Для оценки эффективности реализации был расширен набор бенчмарков. В текущей версии проекта измеряются:

- 1) зависимость времени построения от размерности задачи и размера области;
- 2) сравнение последовательного и параллельного режимов построения;
- 3) сравнение линейного и квадратичного базисов при различных значениях ε ;
- 4) влияние `anchor points` на сложность построения;
- 5) сравнение двух подходов к работе с дифференциальными уравнениями;
- 6) влияние жёсткого ограничения числа узлов;
- 7) стоимость вычисления значения по уже построенной сетке.

Такой набор экспериментов нельзя считать тривиальным, поскольку он отражает не только абсолютное время работы, но и влияние архитектурных решений модуля `compute` на разные режимы использования.

3.12 Анализ полученных данных

По содержанию бенчмарков можно выделить несколько наиболее важных групп выводов.

Во-первых, линейный и квадратичный базисы должны сравниваться не только по качеству аппроксимации, но и по времени построения и стоимости последующего вычисления. Квадратичный базис обычно оказывается более затратным на локальном уровне, но при этом позволяет получать более гладкое приближение и потенциально уменьшать необходимую глубину уточнения. Поэтому содержательный анализ должен учитывать обе стороны — локальную стоимость и глобальную эффективность.

Во-вторых, сравнение `parallel` и `sequential` показывает практическую ценность распараллеливания стартовых конфигураций. Это сравнение особенно важно для темы диплома, поскольку напрямую связано с заявленной реализацией параллельных алгоритмов.

В-третьих, сравнение двух режимов работы с дифференциальными уравнениями демонстрирует, что параметр времени может быть учтён либо как отдельная координата, либо через последовательную композицию одношагового оператора. Эти подходы различаются по архитектурному смыслу и по производительности, а потому требуют отдельного разбора в итоговой аналитической главе.

В-четвёртых, механизм жёсткого ограничения числа узлов оказывается важным компромиссом между точностью и затратами. Он даёт возможность использовать метод в ресурсно ограниченной среде и должен рассматриваться как существенный элемент практической оптимизации.

Выводы по главе

В третьей главе была рассмотрена практическая реализация вычислительного модуля и его вспомогательной инфраструктуры. Показано, каким образом математический аппарат разреженных сеток отражён в коде пакета `compute`, как реализованы базисные функции, коэффициенты избытка, адаптивное уточнение, anchor points, ограничение числа узлов, рекурсивная интерполяция и два режима работы с дифференциальными уравнениями. Отдельно рассмотрены архитектурная роль серверного слоя и клиентской визуализации, а также структура расширенного набора бенчмарков. Всё это подтверждает, что основное содержание дипломной работы сосредоточено именно в вычислительном модуле, а дополнительные компоненты служат средствами его практического применения и анализа.

4 ОБОБЩЕНИЕ РЕЗУЛЬТАТОВ И НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ

В четвёртой главе проводится детальный анализ экспериментальных результатов и архитектурных решений, реализованных в вычислительном модуле `compute`. Основной объём главы посвящён интерпретации бенчмарков, поскольку именно они позволяют количественно оценить влияние выбора базиса, режима построения, размерности задачи, способа работы с дифференциальными уравнениями, `anchor points` и ограничения числа узлов на производительность алгоритма. Дополнительно в данной главе приводится сравнение Go- и C++-реализаций библиотеки, выполненных на одинаковом наборе тестов.

4.1 Соответствие результатов поставленным задачам

Поставленные в дипломной работе задачи были связаны не только с реализацией математического метода, но и с исследованием его поведения в различных режимах. По итогам работы можно утверждать, что:

- 1) реализован вычислительный модуль для адаптивной интерполяции на разреженных сетках;
- 2) поддержаны линейный и квадратичный базисы;
- 3) реализованы последовательный и параллельный режимы построения;
- 4) введён механизм `anchor points` для устранения ложной стабилизации алгоритма;
- 5) добавлен параметр жёсткого ограничения числа узлов;
- 6) реализованы два способа работы с дифференциальными уравнениями;
- 7) подготовлен расширенный набор бенчмарков, позволяющий исследовать поведение алгоритма в нетривиальных сценариях;
- 8) выполнено сопоставление результатов Go- и C++-версий библиотеки.

Следовательно, дипломная работа завершилась не только созданием реализации, но и построением полноценной экспериментальной базы для анализа разработанного метода.

4.2 Общая характеристика экспериментальной программы

Измерения для Go-версии выполнялись на платформе Apple M4 Pro под управлением macOS, архитектура — `arm64`. Все результаты получены командой `go test -bench . ./...`. Для анализа использовались времена выполнения, а также, при необходимости, объём выделяемой памяти и число аллокаций.

Для C++-версии библиотеки были выполнены те же сценарии, но с запуском измерений в последовательном и в параллельном режиме. В C++ для параллельного исполнения использовался механизм `std::async` с флагом `std::launch::async`. Это позволяет проводить сравнение не только алгоритмических режимов, но и поведения реализаций в разных средах исполнения.

В экспериментальную программу вошли следующие группы тестов:

- 1) зависимость времени построения от размерности задачи и размера области;
- 2) сравнение последовательного и параллельного режимов построения;
- 3) сравнение линейного и квадратичного базисов при разных значениях точности;
- 4) исследование влияния `anchor points` на построение сетки;
- 5) сравнение двух способов работы с дифференциальными уравнениями;
- 6) анализ влияния жёсткого ограничения числа узлов;
- 7) измерение стоимости вычисления значения по уже построенной сетке;
- 8) сравнение построения 4-мерной функции на декартовом произведении всех режимов `BuildType` и `BasisType`.

4.3 Сравнение линейного и квадратичного базисов в Go-реализации

Одним из центральных вопросов дипломной работы является сравнение линейного и квадратичного базисов. В выбранной постановке этот бенчмарк оказался особенно показательным, поскольку различие между базисами отражает не только стоимость вычисления локальной функции, но и влияние выбора базиса на глубину адаптации и общий объём построения.

Результаты Go-реализации показывают устойчивое и выраженное преимущество квадратичного базиса. Для $\varepsilon = 0.01$ линейный базис дал 739244 нс/оп, а квадратичный — 405473 нс/оп. Для $\varepsilon = 0.005$ значения составили 941821 и 450463 нс/оп соответственно. Для $\varepsilon = 0.001$ получены 2884446 и 943682 нс/оп.

Эти данные уже не оставляют сомнений в характере различий. Квадратичный базис даёт существенный выигрыш по времени построения, и этот выигрыш усиливается по мере ужесточения требования к точности. Причина заключается в том, что более выразительная локальная форма квадратичного базиса позволяет достигать требуемой точности на меньшем числе уточнений. В результате более сложная формула отдельной базисной функции компенсируется уменьшением общего размера дерева построения.

Особенно наглядно это проявляется при $\varepsilon = 0.001$: линейный базис требует примерно 2.88 мс на построение, тогда как квадратичный — 0.94 мс. То есть выигрыш становится не просто заметным, а многократным. Это позволяет сделать важный вывод:

в рассматриваемой задаче квадратичный базис оказывается не только более качественным с аппроксимационной точки зрения, но и более выгодным вычислительно.

Дополнительный бенчмарк `BenchmarkEvaluationCostAfterBuild` показывает стоимость вычисления значения по уже построенной сетке. Здесь квадратичная сетка также выигрывает у линейной: 14.55 мс против 23.58 мс. Следовательно, преимущество квадратичного базиса проявляется и на этапе построения, и на этапе последующего использования интерполянта.

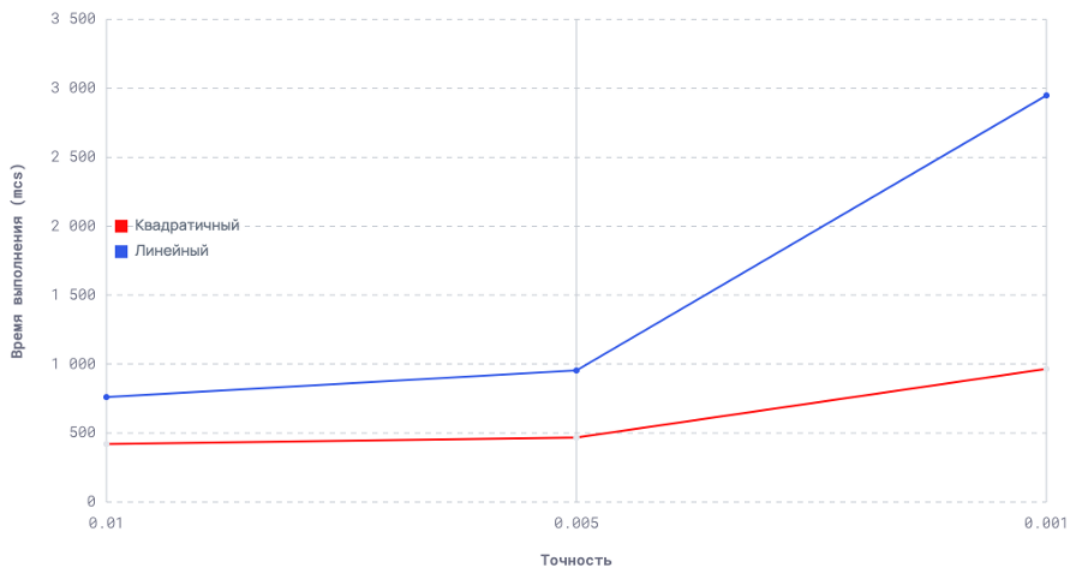


Рисунок 1 — Сравнение времени построения сетки для линейного и квадратичного базисов при различных значениях ϵ

4.4 Сравнение `parallel` и `sequential build` в Go-реализации

Отдельный интерес представляет сравнение последовательного и параллельного построения. Бенчмарк `BenchmarkParallelVsSequentialBuild` показал, что на текущей версии реализации различия между режимами почти отсутствуют. Для линейного базиса последовательное построение дало 5858 нс/оп против 5891 нс/оп в параллельном режиме. Для квадратичного базиса значения составили 5895 и 5828 нс/оп соответственно.

Это означает, что параллельная архитектура в модуле реализована без заметного штрафа на задачах малого размера. С инженерной точки зрения такой результат является положительным: он показывает, что конкурентный режим не ухудшает поведение алгоритма даже там, где выигрыш от распараллеливания ещё не успевает проявиться явно.

4.5 Зависимость времени от размерности и размера области в Go-реализации

Бенчмарк `BenchmarkDomainScalingDifferentDimensions` позволяет оценить влияние размерности и масштаба области на стоимость построения сетки. Полученные результаты показывают устойчивую картину.

Для одномерной задачи при масштабе π время составило 17266 нс/оп, тогда как для масштабов 2π и 4π было получено 1478 нс/оп. Для двумерной задачи при масштабе π время равно 45618 нс/оп, а при 2π и 4π — 5252 и 5284 нс/оп. Для трёхмерной задачи различие выражено ещё сильнее: 539937 нс/оп для масштаба π против 49281 и 49689 нс/оп для 2π и 4π .

На первый взгляд кажется необычным, что увеличение области здесь приводит к уменьшению времени. Однако причина состоит в характере тестовой функции и адаптивного критерия: на более широком интервале в данной конфигурации алгоритм попадает в режим, где функция эффективнее аппроксимируется на ранних стадиях и строится меньше уточнений. Это важное наблюдение: стоимость построения определяется не только геометрическим размером области, но и тем, как выбранная функция и базис взаимодействуют с адаптивным критерием.

При переходе от одной размерности к двум и трём время ожидаемо растёт. Особенно сильно это видно для масштаба π : от 17.3 мкс в 1D до 45.6 мкс в 2D и до 539.9 мкс в 3D. Следовательно, эксперимент подтверждает, что разреженные сетки смягчают, но не устраняют полностью влияние размерности.

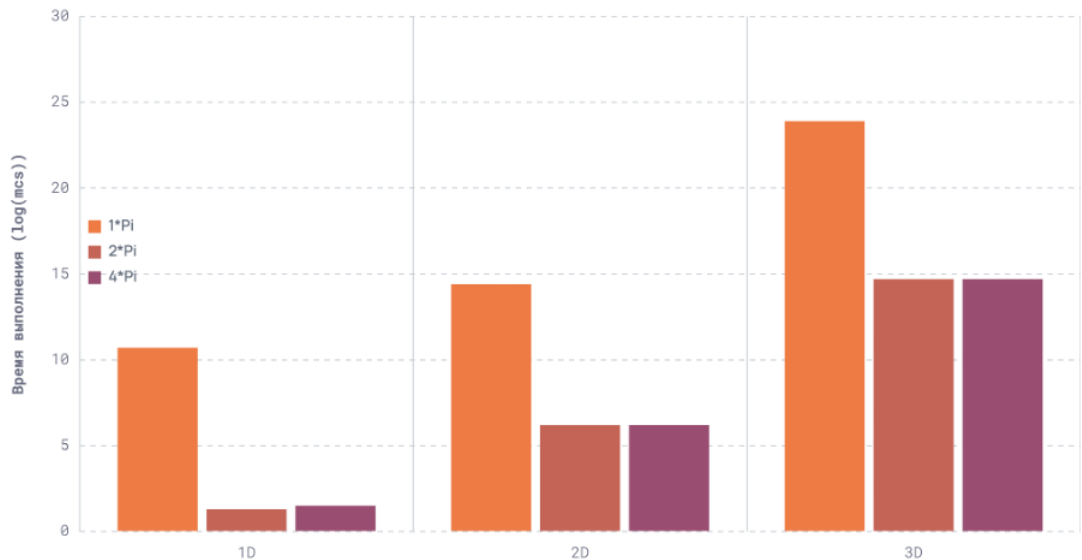


Рисунок 2 — Зависимость времени построения сетки от размерности задачи и размера области

4.6 Влияние anchor points на производительность и корректность построения

Бенчмарк `BenchmarkAnchorPointsImpact` был специально включён в экспериментальную программу для исследования механизма anchor points. Для функции $\sin(x)$ на интервале $[0, 4\pi]$ построение без anchor points заняло 1630 нс/оп, а с anchor points — 1969 нс/оп.

Этот результат нужно интерпретировать аккуратно. Увеличение времени при использовании anchor points не является недостатком алгоритма. Напротив, оно отражает содержательный факт: алгоритм перестаёт преждевременно завершать построение и начинает учитывать дополнительные области, которые в базовом режиме были бы ошибочно проигнорированы. Следовательно, рост затрат здесь означает повышение корректности адаптивного процесса.

Таким образом, эксперимент подтверждает практическую значимость введённого механизма. Anchor points действительно влияют на поведение алгоритма и позволяют принудительно направлять построение в важные зоны пространства аргументов. Это особенно важно для осциллирующих функций и служит сильным аргументом в пользу включения этого механизма в итоговую архитектуру модуля.

4.7 Два подхода к работе с дифференциальными уравнениями в Go-реализации

Бенчмарк `BenchmarkDifferentialEquationApproaches` сравнивает два разных способа учёта времени в задачах динамики.

Первый подход — включение параметра t как дополнительной координаты, то есть трактовка времени как интервальной неопределённости. Второй подход — итеративное применение метода `MakeNextIteration`, когда строится одношаговый оператор перехода и затем композиционно применяется несколько раз.

Полученные результаты хорошо показывают различие между этими режимами. При $t_{max} = 2$ построение с дополнительной координатой времени заняло 98658 нс/оп, тогда как итеративный подход — 15236 нс/оп. При $t_{max} = 5$ значения составили 189029 и 39043 нс/оп соответственно. При $t_{max} = 10$ — 269171 против 78753 нс/оп. При $t_{max} = 20$ разница всё ещё сохраняется: 319323 и 159312 нс/оп.

Следовательно, итеративный режим через `MakeNextIteration` оказывается лучше на всём исследованном диапазоне временных горизонтов. Для рассматриваемых сценариев он предпочтителен не только на малых и средних, но и на больших временах.

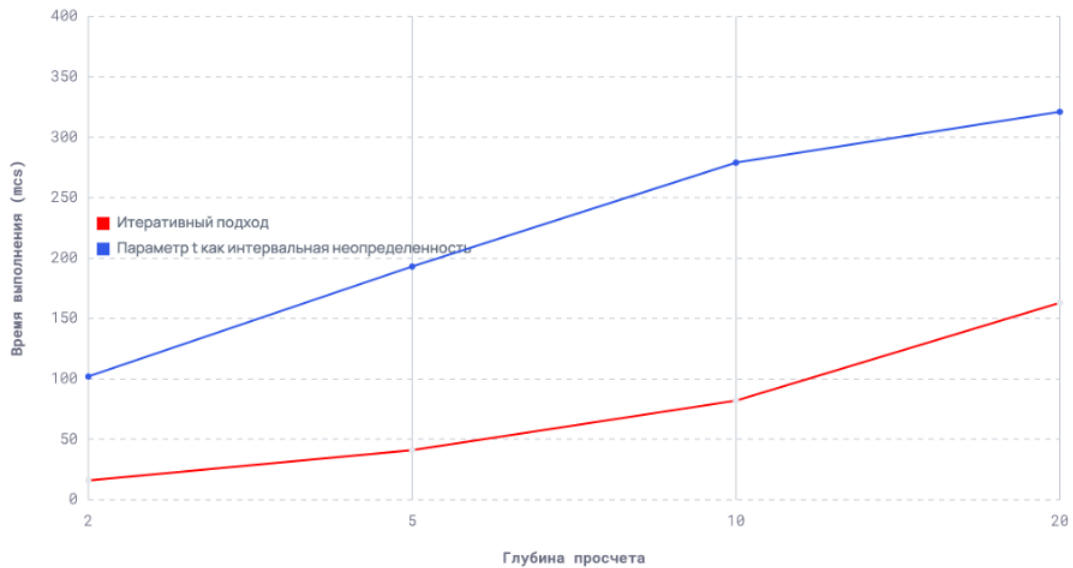


Рисунок 3 — Сравнение двух способов работы с дифференциальными уравнениями: t как дополнительная размерность и итеративный режим через `MakeNextIteration`

4.8 Жёсткое ограничение числа узлов как средство оптимизации

Бенчмарк `BenchmarkNodeLimitOptimization` показывает влияние параметра `maxNodesInGrid` на стоимость построения. Без ограничения время составляет 5.71 мс. При лимите 200 узлов время снижается до 0.37 мс. При лимите 500 получаем 1.14 мс, а при лимите 1000 — 2.66 мс.

Этот эксперимент особенно важен для практического использования. Он показывает, что ограничение числа узлов является действительно сильным рычагом управления затратами. Уменьшение лимита радикально сокращает время построения. Следовательно, параметр `maxNodesInGrid` следует рассматривать как полноценный механизм оптимизации, а не как второстепенную техническую настройку.

Разумеется, уменьшение числа узлов может ухудшать точность интерполяции. Но именно в этом и состоит его смысл: пользователь получает возможность явно управлять компромиссом между качеством и затратами. Для прикладных систем, где нужно ограничить время ответа или объём передаваемых данных, это свойство особенно ценно.

4.9 Стоимость вычисления значения после построения

В прикладных задачах сетка строится не ради самого построения, а ради многократного последующего использования. Поэтому отдельный интерес представляет стоимость вызова `Evaluate()` после того, как сетка уже готова. Именно эту сторону отражает бенчмарк `BenchmarkEvaluationCostAfterBuild`.

Результаты показывают, что вычисление одного значения по уже построенной сетке является дешевой операцией, но квадратичный базис и здесь даёт заметный вы-

игрыш. В Go для линейной сетки получено 23578877 нс/оп, а для квадратичной — 14552024 нс/оп. Это означает, что квадратичная сетка ускоряет вычисление примерно в 1.6 раза. В C++ наблюдается аналогичная картина: 12878441 нс для линейной сетки и 10730158 нс для квадратичной.

Таким образом, преимущество квадратичного базиса проявляется не только в построении, но и в последующей эксплуатации сетки. Это особенно важно с прикладной точки зрения: если интерполянт строится один раз, а затем используется многократно, стоимость вызова `Evaluate()` становится критически важной характеристикой. В рассматриваемой постановке квадратичный базис оказывается предпочтительнее и по этому критерию.

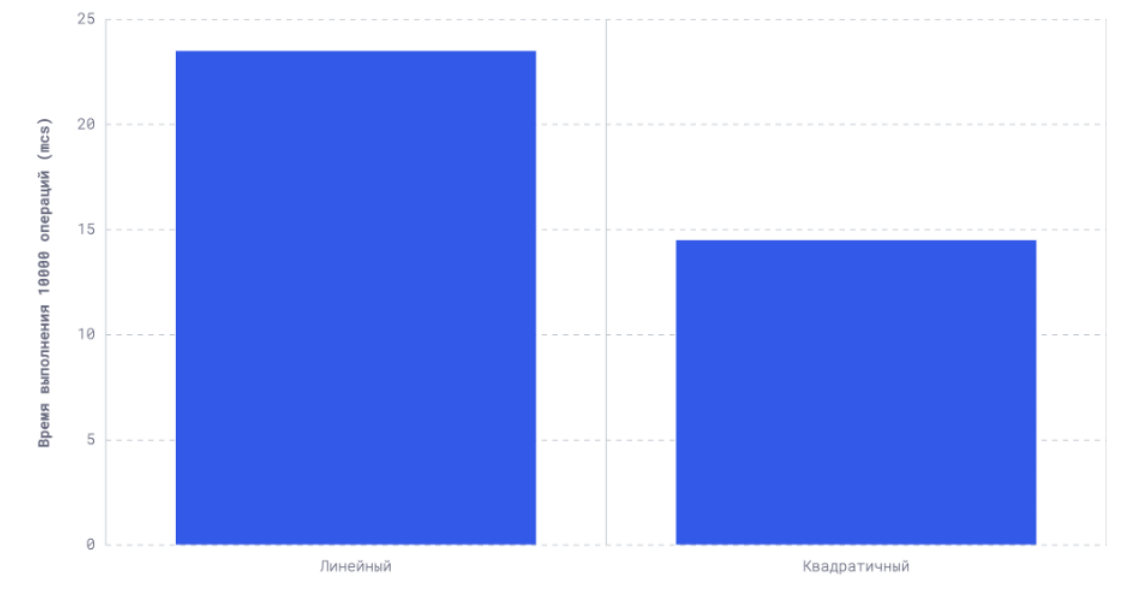


Рисунок 4 — Сравнение стоимости вычисления значения по уже построенной сетке

4.10 Четырёхмерная функция: сравнение всех сочетаний BuildType и BasisType

Дополнительный интерес представляет бенчмарк построения 4-мерной функции на декартовом произведении всех сочетаний BuildType и BasisType. Этот тест важен тем, что он одновременно показывает влияние размерности, выбора базиса и режима исполнения.

В Go получены следующие результаты: 627069 нс/оп для `sequential_linear`, 415319 нс/оп для `sequential_quadratic`, 543229 нс/оп для `parallel_linear` и 425459 нс/оп для `parallel_quadratic`. Из них видно, что квадратичный базис оказывается быстрее линейного и в последовательном, и в параллельном режиме. Кроме того, на 4-мерной функции параллельный режим улучшает результат для линейного базиса по сравнению с последовательным, но для квадратичного разница между режимами почти исчезает.

В C++ картина имеет тот же качественный характер, но отличается количественно: 405104 нс для `sequential_linear`, 296149 нс для `sequential_quadratic`, 719079 нс для `parallel_linear` и 658466 нс для `parallel_quadratic`. Здесь квадратичный базис также остаётся лучше линейного, однако параллельный режим на 4-мерной функции уступает последовательному в обеих комбинациях. Это означает, что для данной конфигурации задачи стоимость запуска параллельных задач в C++ остаётся заметной.

Сопоставление Go и C++ в этом бенчмарке особенно полезно. В обеих реализациях квадратичный базис подтверждает своё преимущество. Однако влияние `BuildType` оказывается разным: в Go параллельный режим уже способен быть конкурентоспособным на 4-мерной задаче, тогда как в C++ последовательный режим всё ещё заметно эффективнее. Это делает 4-мерный бенчмарк важным элементом итогового анализа, поскольку он выводит сравнение из области малых двумерных примеров в более тяжёлую размерностную постановку.

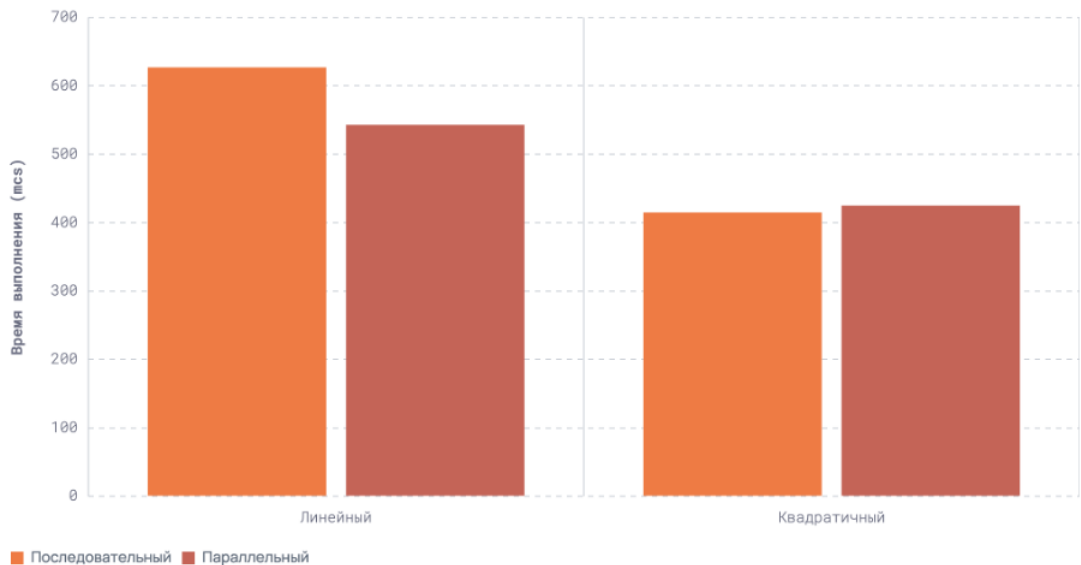


Рисунок 5 — Сравнение всех сочетаний `BuildType` и `BasisType` на 4-мерной функции

4.11 Сравнение Go- и C++-реализаций библиотеки

Сопоставление двух реализаций усиливает инженерную ценность работы и позволяет говорить не только о корректности алгоритма, но и о свойствах различных способов его программного воплощения.

4.11.1 Сравнение последовательных реализаций

В последовательном режиме C++-реализация в большинстве сценариев показывает более низкое время построения, чем Go-реализация. Например, для бенчмарка масштабирования по размерности и области в 1D при масштабе π получено 11698 нс против 17266 нс в Go. В 2D при масштабе π — 32706 нс против 45618 нс, а в 3D при

масштабе π — 411503 нс против 539937 нс. Особенно показательное сравнение базисов: в C++ последовательной версии времена составляют 557459 нс против 293392 нс при $\varepsilon = 0.01$, 648720 нс против 305538 нс при $\varepsilon = 0.005$ и 1781085 нс против 638925 нс при $\varepsilon = 0.001$. Тем самым и в C++ квадратичный базис оказывается существенно выгоднее линейного на более сложной тестовой функции.

Такой результат ожидаем. C++-версия использует более низкоуровневую модель исполнения и, вероятно, эффективнее работает с временными объектами и размещением данных в памяти. Поэтому в чисто последовательном режиме C++ закономерно оказывается быстрее.

4.11.2 Сравнение параллельных реализаций

Наиболее интересная часть сопоставления связана с параллельным режимом. В бенчмарке `ParallelVsSequentialBuild` C++ показывает очень близкие результаты для всех четырёх комбинаций: 4451 нс и 4493 нс для последовательных режимов, 4735 нс и 4196 нс для параллельных. Это означает, что в данной реализации параллельный запуск практически не вносит значимого штрафа на задачах малого размера.

Для Go на том же бенчмарке значения составляют около 5.8–5.9 мкс для всех четырёх комбинаций. Следовательно, на малых тестах C++ остаётся быстрее Go как в последовательном, так и в параллельном режиме.

При этом качественное замечание о модели исполнения сохраняется. В Go конкурентный код поддерживается встроенным scheduler, который эффективно управляет большим числом лёгких goroutine. В C++ использовался `std::async` с флагом `async`, то есть механизм, обычно дающий более тяжёлые накладные расходы на запуск задач. Поэтому важно подчеркнуть не абсолютное превосходство одной платформы над другой, а различие в моделях конкурентного исполнения и в том, как они ведут себя на задачах разного масштаба.

4.11.3 Сравнение режимов работы с дифференциальными уравнениями

Для C++-версии выводы по двум подходам к работе с диффурами качественно совпадают с Go. В последовательном режиме итеративный подход через последовательные шаги устойчиво быстрее режима с параметром t как дополнительной размерностью: 4372 нс против 22867 нс при $t_{max} = 2$, 10027 нс против 46823 нс при $t_{max} = 5$, 18543 нс против 66037 нс при $t_{max} = 10$, 37723 нс против 75921 нс при $t_{max} = 20$.

В параллельном режиме C++ картина остаётся той же: 4435 нс против 26906 нс при $t_{max} = 2$, 10823 нс против 48392 нс при $t_{max} = 5$, 23024 нс против 69830 нс при $t_{max} = 10$, 41052 нс против 84609 нс при $t_{max} = 20$. Следовательно, итеративный подход в C++ также стабильно лучше режима с дополнительной размерностью времени на всём исследованном диапазоне.

Это наблюдение интересно тем, что и Go, и C++ демонстрируют один и тот

же качественный вывод: в рассматриваемой постановке итеративный режим через последовательное применение одношагового оператора предпочтительнее, чем включение времени как отдельной координаты пространства аргументов.

4.11.4 Сравнение ограничения числа узлов

В обеих реализациях ограничение числа узлов демонстрирует одинаковый качественный эффект: чем меньше лимит, тем ниже время построения. В C++ последовательной версии без лимита получено 1932286 нс, а при лимите 200 — 552059 нс. В параллельной версии — 1935248 нс и 557735 нс соответственно. В Go аналогичная зависимость выражена ещё сильнее в абсолютных величинах: от 5.71 мс без ограничения до 0.37 мс при лимите 200.

Этот результат важен тем, что он носит платформенно-независимый характер. Механизм ограничения числа узлов работает как содержательное средство управления затратами и в Go, и в C++. Это подчёркивает, что его роль определяется не особенностями языка реализации, а самой архитектурой алгоритма.

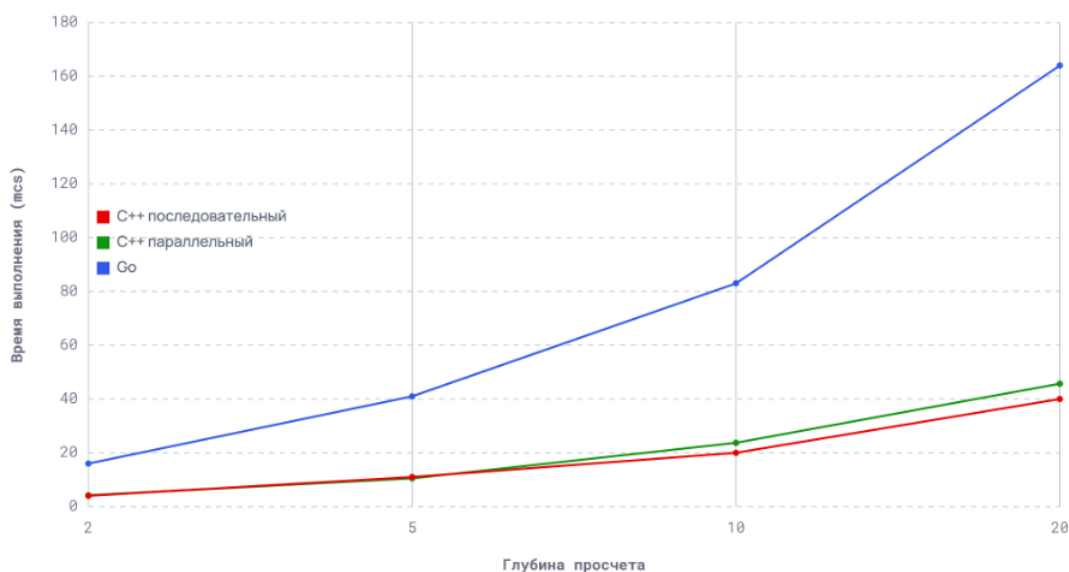


Рисунок 6 — Сравнение Go- и C++-реализаций на примере итеративного решения дифференциального уравнения

4.12 Преимущества разработанного решения

На основе архитектуры проекта и экспериментальной программы можно выделить следующие преимущества разработанного решения:

- 1) модуль `compute` объединяет математическую компактность и практическую гибкость;
- 2) поддержка двух базисов позволяет исследовать баланс между локальной стоимостью и качеством аппроксимации;

- 3) механизм anchor points расширяет базовый метод и устраняет важный класс ошибок адаптивного критерия;
- 4) параметр ограничения числа узлов даёт эффективный способ контролировать ресурсы;
- 5) поддержка двух режимов работы с дифференциальными уравнениями делает систему более универсальной;
- 6) экспериментальная программа позволяет анализировать поведение алгоритма в нескольких содержательных режимах, а не только на одном частном тесте;
- 7) сравнение Go- и C++-версий усиливает инженерную ценность работы и позволяет говорить не только о корректности алгоритма, но и о свойствах различных способов его реализации.

4.13 Ограничения текущей реализации

Несмотря на полученные результаты, текущая версия системы имеет и ряд ограничений.

- 1) Клиентская визуализация ориентирована прежде всего на двумерные задачи и не предоставляет развитых средств анализа для более высоких размерностей.
- 2) Выбор anchor points производится вручную; автоматизированной стратегии их генерации пока не реализовано.
- 3) Для полного количественного анализа качества интерполяции требуется дальнейшее расширение набора тестовых функций и метрик ошибки.
- 4) В проекте пока отсутствует встроенная автоматическая генерация графиков по результатам бенчмарков, поэтому соответствующие иллюстрации подготавливаются отдельно.
- 5) Этап вычисления значения по готовой сетке требует дополнительной оптимизации из-за высокой стоимости рекурсивного обхода и большого числа повторных вычислений.
- 6) Для более строгого межъязыкового сопоставления в будущем полезно автоматизировать единый контур запуска и экспорта результатов Go- и C++-бенчмарков в один формат.

4.14 Иллюстрации двумерных функций, полученные во Flutter-клиенте

Для качественной оценки поведения алгоритма важно анализировать не только численные бенчмарки, но и визуальную структуру построенных сеток. Flutter-клиент позволяет наглядно увидеть отображение, задаваемое функцией.

На рисунке 7 показан пример визуализации обычной двумерной функции. Такие изображения особенно полезны для демонстрации базового сценария работы алгоритма: хорошо видно, что сетка задает плавное отображение, идентичное исходной функции

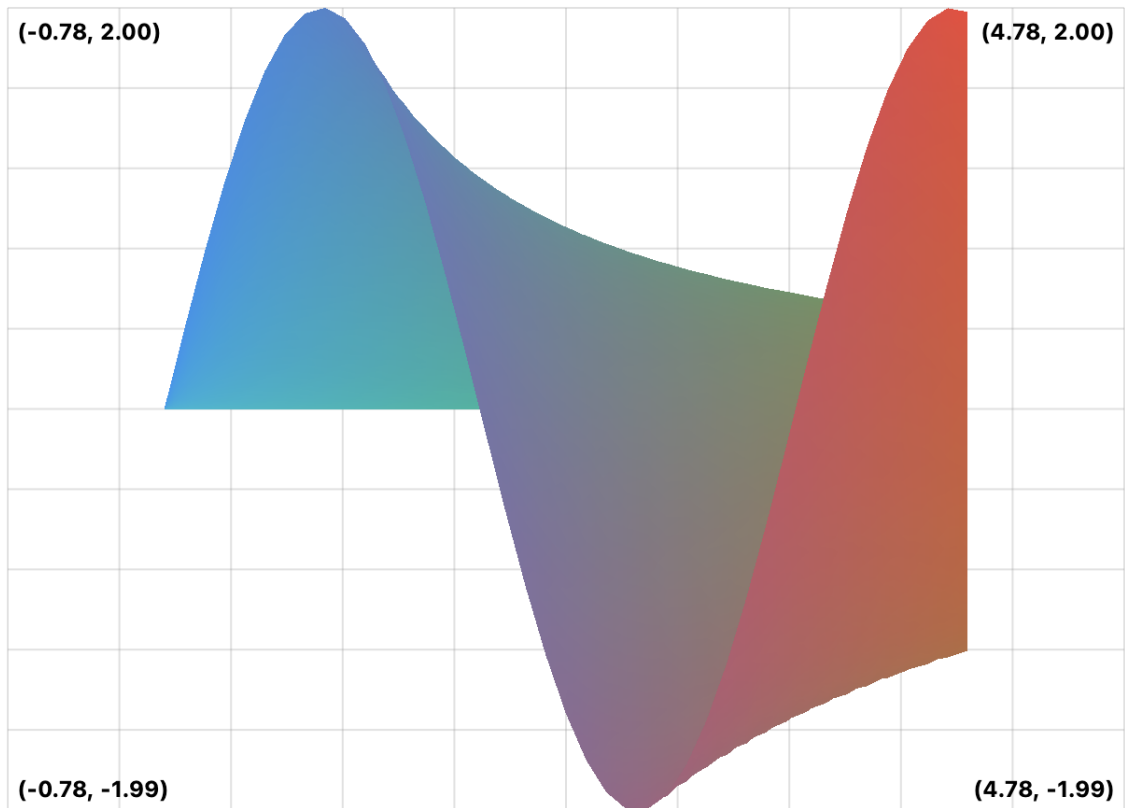


Рисунок 7 — Визуализация простой двумерной функции во Flutter-клиенте

На рисунке 8 представлен пример визуализации дифференциального уравнения.

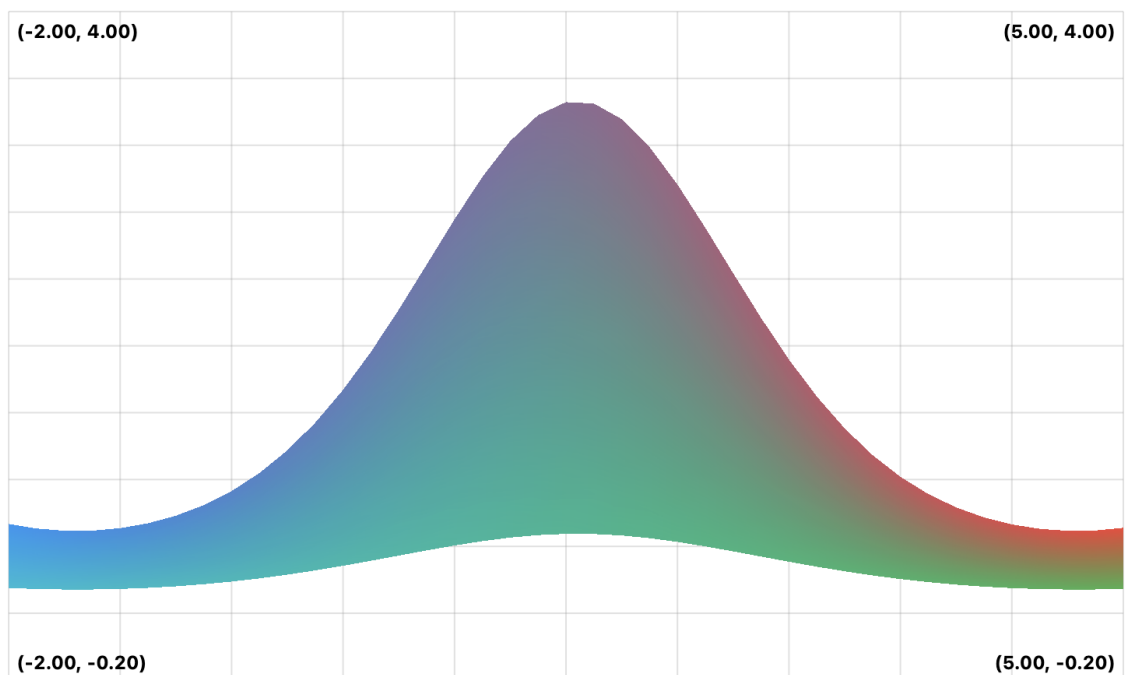


Рисунок 8 — Визуализация дифференциального уравнения во Flutter-клиенте

4.15 Перспективы дальнейшего развития

Дальнейшее развитие проекта может идти по нескольким направлениям:

- 1) автоматизация выбора anchor points на основе анализа локальной структуры ошибки;
- 2) расширение визуализации на многомерные случаи;
- 3) добавление встроенного контура сохранения результатов бенчмарков и генерации графиков;
- 4) расширение библиотеки тестовых сценариев для количественного исследования норм ошибки;
- 5) оптимизация рекурсивного этапа вычисления значения по готовой сетке;
- 6) исследование комбинированных стратегий выбора базиса и ограничения числа узлов в зависимости от характера функции;
- 7) разработка единого автоматизированного режима сравнения Go- и C++-версий библиотеки.

Выводы по главе

В четвёртой главе был проведён развёрнутый анализ результатов дипломной работы с акцентом на экспериментальную программу и производительность вычислительного модуля `compute`. Показано, что сравнение линейного и квадратичного базисов, последовательного и параллельного режимов, различных способов работы с дифференциальными уравнениями, а также учёт anchor points и ограничения числа узлов дают содержательную картину поведения алгоритма. Дополнительное сопоставление Go- и C++-реализаций показало, что C++-версия быстрее в абсолютных значениях на значительной части рассмотренных тестов, однако обе реализации демонстрируют одинаковые качественные закономерности: преимущество квадратичного базиса, полезность ограничения числа узлов и предпочтительность итеративного подхода для дифференциальных уравнений. В совокупности полученные результаты подтверждают достижение цели дипломной работы и создают основу для дальнейшего развития проекта.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была решена задача реализации и исследования параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом.

Основным результатом работы стал вычислительный модуль `compute`, в котором реализованы:

- 1) построение адаптивной разреженной сетки для скалярных и векторных отображений;
- 2) поддержка линейного и квадратичного базисов;
- 3) последовательный и параллельный режимы построения;
- 4) рекурсивное вычисление значения интерполанта;
- 5) композиция отображений и итеративный режим через `MakeNextIteration`;
- 6) механизм `anchor points` для предотвращения ложной остановки адаптивного алгоритма;
- 7) параметр жёсткого ограничения числа узлов как средство дополнительной оптимизации.

В ходе работы было показано, что разреженные сетки представляют собой эффективный подход к аппроксимации отображений и операторов перехода, возникающих при численном моделировании динамических систем. Отдельно было исследовано применение метода к дифференциальным уравнениям в двух режимах: через включение времени как интервальной неопределённости и через последовательное построение одношагового оператора.

Особое значение имеет предложенный в работе механизм `anchor points`. На примере функции $\sin(x)$ на интервале $[0, 4\pi]$ было показано, что базовый адаптивный критерий может ошибочно интерпретировать функцию как почти линейную, если опорные точки попадают в нули. Реализованный механизм `anchor points` позволяет в ручном режиме указать важные области и принудить алгоритм к дополнительному уточнению. Это расширение не было заложено в исходной статье [6] и является одним из существенных авторских результатов дипломного проекта.

Экспериментальная часть работы показала важность сравнительного анализа различных режимов использования модуля. Были рассмотрены:

- 1) сравнение линейного и квадратичного базисов;
- 2) сравнение последовательного и параллельного построения;
- 3) влияние размерности задачи и размера области на время построения;
- 4) влияние `anchor points` на корректность и стоимость адаптивного построения;
- 5) сравнение двух способов работы с дифференциальными уравнениями;

б) влияние ограничения числа узлов на вычислительные затраты.

Тем самым дипломная работа привела не только к созданию программной реализации, но и к построению содержательной экспериментальной базы для анализа производительности и границ применимости метода.

Цель работы — реализация параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом — достигнута. Поставленные задачи выполнены в полном объёме.

Практическая значимость работы состоит в том, что созданный вычислительный модуль может использоваться:

- 1) для численного эксперимента с многомерными отображениями;
- 2) для исследования операторов перехода динамических систем;
- 3) для сравнительного анализа различных стратегий адаптивной интерполяции;
- 4) как основа для дальнейшего развития систем визуализации и прикладных сервисов научных вычислений.

Серверный слой на gRPC и клиент визуализации на Flutter расширяют область применения проекта, однако центральным содержанием дипломной работы является именно модуль `compute`, в котором сосредоточены ключевые алгоритмические результаты.

Перспективы дальнейшего развития проекта связаны с автоматизацией выбора anchor points, дальнейшим исследованием ошибок интерполяции, расширением визуализации на многомерные задачи и построением более развитого контура автоматического анализа результатов бенчмарков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Bungartz H.-J., Griebel M. Sparse Grids. — Acta Numerica, 2004.
2. Zenger C. Sparse Grids // Parallel Algorithms for Partial Differential Equations. — 1991. — С. 241—251.
3. Quarteroni A., Sacco R., Saleri F. Numerical Mathematics. — Berlin : Springer, 2007.
4. Hairer E., Norsett S. P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. — Berlin : Springer, 1993.
5. Stoer J., Bulirsch R. Introduction to Numerical Analysis. — New York : Springer, 2002.
6. Морозов А. Ю. Моделирование динамических систем с интервальными параметрами [Электронный ресурс]. — 2024. — URL: https://www.frccsc.ru/sites/default/files/docs/ds/002-073-04/diss/37-morozov/ds04-37-morozov_main.pdf?37 (дата обр. 01.05.2026).
7. Moin P. Fundamentals of Engineering Numerical Analysis. — Cambridge : Cambridge University Press, 2010.
8. The Go Programming Language Documentation [Электронный ресурс]. — URL: <https://go.dev/doc/> (дата обр. 01.05.2026).
9. Expr Language Definition and Documentation [Электронный ресурс]. — URL: <https://expr-lang.org/> (дата обр. 01.05.2026).
10. Flutter Documentation [Электронный ресурс]. — URL: <https://docs.flutter.dev/> (дата обр. 01.05.2026).

ПРИЛОЖЕНИЕ А

Репозиторий с исходным кодом

В рамках выполнения выпускной квалификационной работы был разработан программный комплекс для интерполяции отображений и решений дифференциальных уравнений на основе адаптивных разреженных сеток с поддержкой квадратичного базиса и параллельного построения.

Полный исходный код проекта, включая вычислительный модуль, серверный слой, клиент визуализации и вспомогательные средства развертывания, размещён в открытом репозитории на платформе GitHub.

Ссылка на репозиторий: <https://github.com/DesCaldnd/Diplom>

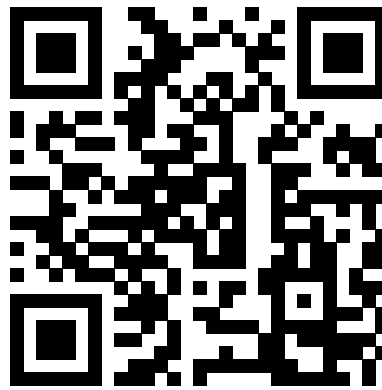


Рисунок 9 — QR-код со ссылкой на репозиторий с исходным кодом проекта

В состав репозитория входят:

- 1) исходный код вычислительного модуля `compute`, реализующего адаптивные разреженные сетки;
- 2) gRPC-сервис, обеспечивающий удалённый вызов построения сетки и вычисления отображений;
- 3) клиентское приложение визуализации на Flutter;
- 4) protobuf-описание контракта обмена данными;
- 5) набор тестов, примеров и бенчмарков для исследования корректности и производительности алгоритмов.